阶梯代码评测集设计说明

L1–L5 五档难度：正确性、效率与代码质量

2026-10-27 codingBench `cb ladder`

摘要

现有的 LiveCodeBench 最新 35 题已被一线模型做满：复核判分后 DeepSeek-V4.1-Flash 与 DeepSeek-V4-Pro 均为 35/35，Kimi K3.1 Pro 为 34/35，正确率已无法区分模型；同一批答案的运行时间却相差明显（相对最快答案的几何平均倍数为 1.29、2.00、2.74）。为此我们构建了一套按人类难度分为五档的测试集：

- **L1–L4**：USACO 2024–25 与 2025–26 两个赛季的铜、银、金、铂金组，L3、L4 另加入 2026 年的 EGOI 与 IOI 新题；
- **L5**：FrontierCS 算法赛道，没有已知最优解，按 0–100 给连续分。

所有题目、测试数据和题解都来自官方发布。每道题同时报告四类结果：

- 宽时限下的正确性；
- 官方时限下的正确性；
- 相对官方解的运行时间与内存；
- 相对官方解的时间、空间复杂度增长阶（是否用了更优或更差的解法）；
- Python 档位的静态代码质量。

评测流程已实现为一条命令（`cb ladder`）：接入模型 API 后自动生成、判题、复测性能并输出报告。

1 背景与目标

1. **恢复区分度。**题目要足够难，并且难度分层：弱模型在低档拉开差距，强模型在高档拉开差距。
2. **不止看对错。**同时测“够不够快”，即官方时限下能否通过、运行时间是官方解的多少倍，以及内存、代码质量、调用成本与时延。
3. **防止训练数据泄露。**记录每道题的比赛日期，并单独报告 2026 年新题的成绩作为对照。
4. **可复现。**只用官方测试数据和官方题解，来源钉在固定的版本上，判题规则与官方一致。

2 设计原则

- **难度用人类标尺划分。**USACO 的四个组别由真实选手的晋级线划定（例如 2026 年第一场金组得分 800 以上晋级铂金），EGOI、IOI 是国际奥赛，所以档位对应的是人类选手的水平，而不是哪个模型觉得难。
- **只用官方数据。**AtCoder 自 2024 年 12 月起停止公开测试数据，其 robots.txt 也禁止抓取提交记录，难题的官方题解里多半没有代码；LeetCode 的测试不公开，用户协议禁止批量抓取。这两个来源都给不出可信的标准答案，因此没有采用。
- **高档用连续分。**USACO 按测试点给部分分，EGOI、IOI 按子任务给分，FrontierCS 按 checker 给 0–100 的连续分，高档位不会像通过率那样很快饱和。

- **正确与速度分开测。**每个测试点只运行一次，时限放宽到官方的 5 倍，再按实际 CPU 时间同时得出两种判定：宽时限判的是算法对不对，官方时限判的是够不够快。
- **先验证判题本身。**建库时先用我们的判题程序跑官方题解；没有任何一份官方解能全部通过的题直接剔除，这同时检验了测试数据、比对规则和判题程序。

3 题库组成

档位	来源	题数	比赛时间	作答语言	计分
L1 基础	USACO 铜组	21	2024-12 至 2026-02	Python	测试点等分
L2 中等	USACO 银组	21	2024-12 至 2026-02	Python	测试点等分
L3 困难	USACO 金组	21	2024-12 至 2026-02	C++	测试点等分
	EGOI 2026（2026 子集）	8	2026-05	C++	子任务（取最低）
L4 顶尖	USACO 铂金组	21	2024-12 至 2026-02	C++	测试点等分
	IOI 2026 批处理题（2026 子集）	2	2026-08	C++	子任务（取最低）
L5 开放式	FrontierCS 算法赛道（按类别抽样）	30	题库 v1.1	C++17	0–100 连续分

- USACO 每个赛季有若干场比赛，每场每组 3 题。2024–25 赛季取 12 月、1 月、2 月和美国公开赛共 4 场，2025–26 赛季取 1 月、1 月底和 2 月 3 场线上赛（3 月 28 日的公开赛为线下监考赛，官网上没有它的题目和测试数据），合计 84 题，四个组别各 21 题。新赛季开始后可以一键补入。
- IOI 2026 有 6 题，其中 4 题是“通信”题，需要管理程序与多个选手进程交互，现在的判题程序还不支持，暂未列入；先纳入 2 道批处理题（Monuments、Magic City）。
- FrontierCS 题库有 180 道开放式难题，固定抽 30 题，按题目类别均匀抽取；需要时可以跑全部 180 题。
- 表中题数是建库前的计划数。建库校验时，没有官方解能通过的题，以及有多个合法答案却没有 checker 的题，都会被剔除，最终以 `samples/ladder.json` 为准。
- 原始数据共约 4.7 GB。L1–L4 的题面、测试数据与官方题解约 2.6 GB，FrontierCS 约 2.2 GB。题面和测试数据只存在服务器上，不进仓库、不公开；仓库里只记录题号、档位、日期和时限。

4 判题与计分原理

4.1 运行环境

判题在服务器本机进行，不依赖 Docker。

- **运行身份与限制。**答案以无特权用户 `bench` 运行，经一个小的启动器设置五项上限：
  - 地址空间：内存上限加 64 MB；
  - 栈：等于内存上限；
  - CPU 时间；
  - 输出大小：256 MB；

  - 进程绑定在一个独占的核上。
- **计时**。CPU 时间和峰值内存取自子进程本身的资源统计（`wait4`）。
- **Python 档**。用 CPython 3.11，只允许标准库。官方时限是 USACO 给 Python 的时限，默认 4 秒，是 C++ 的两倍；题目注明加时的，按比例放大。
- **C++ 档**。编译命令为 `g++ -O2 -std=gnu++17`。IOI 题与官方 grader 一起编译，参数与 IOI 2026 的 CMS 一致（`-DEVAL -std=gnu++20 -O2`，不静态链接）。

4.2 判定与比对

- 每个测试点的结果是以下之一：
  - 正确（含部分分）；
  - 答案错误；
  - 超时；
  - 超内存；
  - 运行错误；
  - 编译错误。

  宽时限为官方时限的 5 倍（`LADDER_LENIENT`）。在宽时限内通过、但 CPU 时间超过官方时限的测试点，在“官方时限”口径下记 0 分。
- **USACO**：忽略空白、逐个 token 比对，与 USACO 的规则一致。
- **EGOI**：用官方 checker（CMS 约定，checker 输出 0–1 的得分）。
- **IOI**：官方 grader 与选手代码一起编译，输出交给官方 checker。
- **FrontierCS**：用官方 testlib checker 或交互器。规则照搬官方的 `judge_engine.js`：
  - 只解析 checker 自己输出的 `Ratio`；
  - 退出码 0 表示正确，7 表示部分分；
  - 达到参考解得 60 分、超过参考解按对数加分，这部分写在各题的 checker 里。

4.3 计分

- **每题得分（0–100）**按来源的官方规则计算（公式见第 5 节）：
  - USACO：每个测试点分值相同；
  - EGOI、IOI：子任务取其中所有测试点的最低分，再乘以该子任务的分值（CMS GroupMin）；
  - FrontierCS：所有测试点 Ratio 的平均值乘以 100。
- **解出**指每个测试点都满分。
- **采样**：每档的采样次数为 L1、L2、L5 各 1 次，L3、L4 各 3 次（`LADDER_N`），temperature 0.2。高档题量少，多采样能减小单次运气的影响。pass@1 是所有采样中解出的比例；pass@n 是至少有一次解出的题所占比例。
- **提示词**：USACO、EGOI 沿用 LiveCodeBench 的写法，并给出时限和内存限制；IOI 要求只实现规定的函数；FrontierCS 用官方生成脚本的原始提示词，便于和它的公开排行榜对照。

4.4 性能对照

建库时，每题的所有官方题解都要在全部测试点上判一遍；能全部通过的官方解中，最快的一份作为该题的性能对照。

评测时，对每份解出的答案：

- 取官方解最慢的 3 个测试点；
- 答案与官方解交替各运行 3 次，取 CPU 时间的中位数；
- 报告“答案 CPU 时间 ÷ 官方解 CPU 时间”，各档取几何平均。

只有官方解在测试点上至少需要 0.05 秒 CPU 的题才做这项对照，否则时间主要是进程启动开销。Python 档只与 Python 官方解比较，没有 Python 官方解的题不做这项对照。

4.5 时间与空间复杂度

正确率只说明“做没做对”，复杂度还能说明“用了什么样的解法”。一份答案如果增长得比官方解还慢，往往意味着更好或更新颖的算法；增长得更快，则说明它虽然在这批测试上通过了，但算法在渐近意义上更差。

方法参照 BigO(Bench)[10] 一类基于规模扫描的复杂度推断，不同的是不另外生成输入，直接用官方测试：

- 每个测试点的输入字节数作为规模 x；
- 答案在该测试点上的 CPU 时间（或峰值内存）扣除同语言空程序的开销，作为 y；
- 在 y 高于噪声下限的测试点上，拟合 $\log y = a + k \log x$，斜率 k 就是经验增长幂次；
- 在答案与官方解都可测的测试点上，拟合 $\log(y_{\text{答案}}/y_{\text{官方}})$ 对 $\log x$ 的斜率 Δk：$\Delta k \le -0.3$ 记为更优，$|\Delta k| < 0.3$ 记为相同，$\Delta k \ge 0.3$ 记为更差。

输入大小变化不足 8 倍，或可测的测试点少于 4 个时，规模和真实的 n 关系不大（例如输入只有一个 $N \le 10^{18}$ 的数），记为不可测。时间和空间分别判定。增长慢于官方解的答案会在报告中单独列出，供人工复核是否真的是更优的解法。

5 评测维度与打分标准

各维度分开报告，不合成总分；表格中 ↑ 越大越好，↓ 越小越好，没有箭头的只作描述。

维度	指标	计算方法	方向
正确性	测试点得分 s_t	每个测试点 0–1：USACO 忽略空白逐 token 比对，相同为 1、否则 0；EGOI、IOI 取官方 checker 给出的 0–1 分；FrontierCS 取官方 checker 输出的 Ratio（达到参考解为 0.6，超过参考解按对数加分，最高 1）。程序编译失败、运行出错、超时或超内存，该测试点为 0。	
正确性	题目得分 S（0 – 100）	USACO：$S = 100 \cdot \frac{1}{T}\sum_t s_t$（每个测试点等分）；EGOI、IOI：$S = 100 \cdot \frac{\sum_g w_g \min_{t \in g} s_t}{\sum_g w_g}$（子任务 g 取其中测试点的最低分，再乘以子任务分值 w_g）；FrontierCS：$S = 100 \cdot \frac{1}{T}\sum_t \text{Ratio}_t$。	↑
正确性	解出	所有测试点 $s_t = 1$。	

维度	指标	计算方法	方向
正确性	pass@1、平均得分、pass@n	pass@1 = 解出的采样数 ÷ 采样总数 × 100；平均得分 = 所有采样的 S 的平均；pass@n = 至少一次采样解出的题数 ÷ 题数 × 100。每档采样次数：L1、L2、L5 各 1 次，L3、L4 各 3 次，temperature 0.2。	↑
正确性 / 效率	宽时限与官方时限	每个测试点只运行一次，时限为官方时限的 5 倍（宽时限）。宽时限口径：按上面的规则计分。官方时限口径：CPU 时间超过官方时限的测试点记 0，再按同样的规则算 S、pass@1。前者回答“算法对不对”，后者回答“是否和人类选手一样在时限内通过”。	↑
效率	运行时间倍数	只对解出的答案、且官方解在某个测试点上至少需要 0.05 秒 CPU 的题：取官方解最慢的 3 个测试点，答案与官方解交替各运行 3 次取中位数，r = 答案 CPU 时间 ÷ 官方解 CPU 时间（取官方解最慢的那个测试点）；各档报告 r 的几何平均和题数。同语言比较。	↓
效率	内存倍数	解出的答案在所有测试点上的峰值内存 ÷ 官方解的峰值内存，各档取几何平均。	↓
复杂度	增长幂次 k 与 Δk	以测试点输入字节数为规模 x，CPU 时间（或峰值内存）扣除空程序开销为 y，在 y 高于噪声下限（10 ms / 2 MB）的测试点上拟合 $\log y = a + k \log x$；在答案与官方解都可测的测试点上拟合 $\log(y_{答案}/y_{官方})$ 对 $\log x$ 的斜率 Δk。$\Delta k \le -0.3$ 为更优（增长慢于官方解，列出供人工复核），$\lvert\Delta k\rvert < 0.3$ 为相同，$\Delta k \ge 0.3$ 为更差；输入大小变化不足 8 倍或可测测试点少于 4 个记为不可测。时间、空间分别判定。	
失败类型	每份未解出答案的主要原因	按顺序取第一个成立的：无代码、编译错误、答案错误、运行错误、超内存、超时、部分分（部分测试点得分）；宽时限下解出但官方时限下没有全部通过的记为“只在官方时限下超时”。	
静态质量	Pylint、Radon、SonarQube（只限 L1–L2 的 Python 答案）	按 arXiv:2610.19167 的口径与版本：Pylint 3.3.5 默认配置的问题数（C/R/W/E）与 0–10 分，Radon 6.0.1 的圈复杂度（所有条目之和）、可维护性指数 MI、SLOC，SonarQube 10.4 的可维护性/可靠性/安全问题数；报告正确答案的每份平均与每百行 SLOC 的问题数。	见表头
成本与时延	token、费用、延迟、首个 token 时间	费用 = (输入 − 缓存命中) × 输入单价 + 缓存命中 × 命中单价 + 输出（含思考）× 输出单价，单价按每百万 token，分时计价按每次调用的开始时间；延迟为每次调用从发出到收完的时间。	↓
泄露检查	2026 子集与 USACO 子集	L3、L4 中 EGOI/IOI 2026 新题与同档 USACO 旧题分开报告 pass@1 与平均得分；新题明显更差时提示旧题可能见过。只作提示，不作结论。	

6 预期结果

以下是根据已有数据作出的预测，不是测量结果；它们的作用是给出判断标尺，实际数字以评测为准。

档位	一线模型的预期	依据
L1	pass@1 95% 以上	难度与 LiveCodeBench 中等题相当，现有模型在 LCB 上已做满
L2	80–100%；官方时限下略低	Python 运行较慢，USACO 提示高组别的题 Python 未必能在时限内通过
L3	USACO 金组 50–85%；EGOI 更低	EGOI 的最后几题接近 IOI 难度
L4	pass@1 15–50%，平均得分 40–75	2026 年第一场铂金组只有 2 名认证选手超过 500/1000 分；OpenAI 的推理系统在 IOI 2025 线上赛达到金牌分数、与人类同场排名第 6 [12]，但那是 50 次提交、有评测反馈的条件，我们是单次作答、没有反馈
L5	40–57 分	FrontierCS 2026-10-18 公布的 6 个一线模型为 47.8–56.5 分，人类专家 86.99 [4]
性能	运行时间约为官方解的 1–3 倍	此前 LCB 35 题中三个模型相对最快答案为 1.29–2.74 倍

- **区分度**主要来自 L3–L5：低档用来确认基本能力，同时测小模型。
- **两种时限的差距**预计在 Python 档（L2）最明显，C++ 档较小。如果某模型在宽时限下解出、官方时限下失败的题特别多，说明它常写出思路正确但复杂度偏高的解法。
- **如果 USACO 子集明显好于同档的 2026 子集**，提示旧题可能出现在训练数据里。但两部分的难度分布不完全相同，只能作为提示，不能作为结论。
- **成本**：一次完整评测约 228 次调用，包括 L1 21 次、L2 21 次、L3 87 次、L4 69 次、L5 30 次。按此前 LCB 难题的单次费用粗估：
  - DeepSeek-V4.1-Flash 约 $2.5；
  - DeepSeek-V4-Pro 约 $22；
  - Kimi K3.1 Pro 约 $56。

  高档题思考更长，实际可能更高。

7 局限与后续工作

- **题目时效。**USACO 题目截至 2026 年 2 月，可能已进入部分模型的训练数据，所以记录了比赛日期并设 2026 子集作对照。USACO 2026–27 赛季公布后补入新题。
- **IOI 覆盖不全。**IOI 2026 的 4 道通信题暂不支持，下一步实现“管理程序 + 多个选手进程”的判题。
- **C++ 档没有静态质量表。**所用工具只支持 Python，SonarQube 10.4 社区版不分析 C/C++。
- **时限照搬官方。**评测机与 USACO、EGOI、IOI 官方评测机的速度不同。时限沿用官方值，建库时会确认官方解在这台机器上能在官方时限内通过。
- **L5 判题程序待验证。**FrontierCS 官方用 go-judge 容器判题，我们按其规则实现了本机判题程序。下一步重判它仓库里公开的历史解，与公开排行榜核对。
- **AlgoTune 性能专项未上线。**它测“把数值程序写快”，依赖约 3–4 GB 的软件包，等服务器网络恢复后再上线。
- **服务器网络很慢。**服务器入口带宽实测约 350 KB/s，第一跳网关就有 25% 丢包，首次下载全部数据约需 4 小时。

8 使用方法

```
cb ladder-data                # 后台下载（可断点续传）；--status 看进度
cb ladder-build               # 整理题目；--calibrate 判官方解并选出性能对照
export MODEL=... OPENAI_BASE_URL=... OPENAI_API_KEY=...
cb ladder --smoke             # 每档 1 题，先确认接口
cb ladder                     # 全量：生成 -> 判题 -> 性能复测 -> 复杂度 -> 静态质量 -> 报告
cb ladder-report <运行A> <运行B> --out <目录>   # 多模型对比报告
```

管路的测试 tests/test_ladder.py 用合成题和一个假的模型接口，把生成、判题（四种判题方式、超时、超内存、两种时限）、性能复测和报告完整跑一遍，在 CI 和服务器上均已通过。

参考文献

[1] USACO 2024–25、2025–26 赛季比赛结果、题面、测试数据与题解. https://usaco.org/

[2] EGOI 2026 题目包. https://github.com/egoi-org/egoi-2026（commit 08eedd3）

[3] IOI 2026 题目档案（CC BY 4.0）. https://github.com/ioi/task-archive（commit 70278df）

[4] Q. Mang, W. Chai, Z. Li, et al. FrontierCS: Evolving Challenges for Evolving Intelligence. arXiv:2512.15699, 2025. 数据与结果：https://github.com/FrontierCS/Frontier-CS（commit 12ab268）

[5] N. Jain, K. Han, A. Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974, 2024.

[6] Q. Shi, M. Tang, K. Narasimhan, et al. Can Language Models Solve Olympiad Programming? arXiv:2404.10952, 2024.

[7] R. Qiu, W. W. Zeng, J. Ezick, et al. How Efficient is LLM-Generated Code? A Rigorous & High-Standard Benchmark. arXiv:2406.06647, 2024.

[8] N. M. Le, Y. Xu, Z. Wang, et al. Rethinking Code Performance Benchmarks for LLMs. arXiv:2607.07619, 2026.

[9] J. Huang, T. Mahmud, M. Che, et al. Beyond Functional Correctness: Static Code Quality Profiling of LLM-Generated Code. arXiv:2610.19167, 2026.

[10] P. Chambon, B. Rozière, B. Sagot, G. Synnaeve. BigO(Bench): Can LLMs Generate Code with Controlled Time and Space Complexity? arXiv:2503.15242, 2025.

[11] O. Press, B. Amos, H. Zhao, et al. AlgoTune: Can Language Models Speed Up General-Purpose Numerical Programs? arXiv:2507.15887, 2025.

[12] OpenAI. 关于 IOI 2025 线上赛成绩的公告, 2025-08-11. https://x.com/OpenAI/status/1954969035713687975